



DEPARTMENT OF
COMPUTER SCIENCE

ALEXANDRE MIGUEL DA SILVA PERES

BSc in Computer Science and Engineering

PLATFORM FOR DISSERTATION/PROJECT PROPOSAL AND MANAGEMENT

Dissertation Plan
MASTER IN COMPUTER SCIENCE AND ENGINEERING
SPECIALIZATION IN PROGRAMMING LANGUAGES AND SOFTWARE SYSTEMS

NOVA University Lisbon

Draft: January 21, 2026



PLATFORM FOR DISSERTATION/PROJECT PROPOSAL AND MANAGEMENT

ALEXANDRE MIGUEL DA SILVA PERES

BSc in Computer Science and Engineering

Adviser: Nuno Preguiça

Full Professor, NOVA University Lisbon

Co-adviser: Vítor Duarte

Assistant Professor, NOVA University Lisbon

Dissertation Plan
MASTER IN COMPUTER SCIENCE AND ENGINEERING
SPECIALIZATION IN PROGRAMMING LANGUAGES AND SOFTWARE SYSTEMS

NOVA University Lisbon

Draft: January 21, 2026

ABSTRACT

Regardless of the language in which the dissertation is written, usually there are at least two abstracts: one abstract in the same language as the main text, and another abstract in some other language.

The abstracts' order varies with the school. If your school has specific regulations concerning the abstracts' order, the NOVAthesis L^AT_EX (`novathesis`) (L^AT_EX) template will respect them. Otherwise, the default rule in the `novathesis` template is to have in first place the abstract in *the same language as main text*, and then the abstract in *the other language*. For example, if the dissertation is written in Portuguese, the abstracts' order will be first Portuguese and then English, followed by the main text in Portuguese. If the dissertation is written in English, the abstracts' order will be first English and then Portuguese, followed by the main text in English. However, this order can be customized by adding one of the following to the file `5_packages.tex`.

```
\ntsetup{abstractorder={<LANG_1>,...,<LANG_N>}}  
\ntsetup{abstractorder={<MAIN_LANG>={<LANG_1>,...,<LANG_N>}}}
```

For example, for a main document written in German with abstracts written in German, English and Italian (by this order) use:

```
\ntsetup{abstractorder={de={de,en,it}}}
```

Concerning its contents, the abstracts should not exceed one page and may answer the following questions (it is essential to adapt to the usual practices of your scientific area):

1. What is the problem?
2. Why is this problem interesting/challenging?
3. What is the proposed approach/solution/contribution?
4. What results (implications/consequences) from the solution?

Keywords: One keyword · Another keyword · Yet another keyword · One keyword more
· The last keyword

RESUMO

Independentemente da língua em que a dissertação está escrita, geralmente esta contém pelo menos dois resumos: um resumo na mesma língua do texto principal e outro resumo numa outra língua.

A ordem dos resumos varia de acordo com a escola. Se a sua escola tiver regulamentos específicos sobre a ordem dos resumos, o template (L^AT_EX) *novathesis* irá respeitá-los. Caso contrário, a regra padrão no template *novathesis* é ter em primeiro lugar o resumo *no mesmo idioma do texto principal* e depois o resumo *no outro idioma*. Por exemplo, se a dissertação for escrita em português, a ordem dos resumos será primeiro o português e depois o inglês, seguido do texto principal em português. Se a dissertação for escrita em inglês, a ordem dos resumos será primeiro em inglês e depois em português, seguida do texto principal em inglês. No entanto, esse pedido pode ser personalizado adicionando um dos seguintes ao arquivo `5_packages.tex`.

```
\abstractorder(<MAIN_LANG>):={<LANG_1>,...,<LANG_N>}
```

Por exemplo, para um documento escrito em Alemão com resumos em Alemão, Inglês e Italiano (por esta ordem), pode usar-se:

```
\ntsetup{abstractorder={de={de,en,it}}}
```

Relativamente ao seu conteúdo, os resumos não devem ultrapassar uma página e frequentemente tentam responder às seguintes questões (é imprescindível a adaptação às práticas habituais da sua área científica):

1. Qual é o problema?
2. Porque é que é um problema interessante/desafiante?
3. Qual é a proposta de abordagem/solução?
4. Quais são as consequências/resultados da solução proposta?

Palavras-chave: Primeira palavra-chave · Outra palavra-chave · Mais uma palavra-chave · A última palavra-chave

CONTENTS

Acronyms	xi
1 Introduction	1
1.1 Context and Background	1
1.2 Problem Statement	1
1.3 Objectives	3
1.4 Methodology	3
2 State of the art and Technologies	5
2.1 Backend Architectures and BaaS	5
2.1.1 Supabase (Backend-as-a-Service)	6
2.1.2 Custom Backend: Spring Boot	6
2.1.3 Custom Backend: Express.js (Node.js)	7
2.2 Data Persistence: SQL vs. NoSQL	7
2.2.1 The Relational Model (SQL)	7
2.2.2 The Non-Relational Model (NoSQL)	8
2.2.3 Justification for the Chosen Approach (PostgreSQL)	9
2.3 Frontend Frameworks and User Experience	9
2.4 AI-Assisted Development (Vibe Coding)	9
3 Requirements and Current System Analysis	11
3.1 Analysis of the Legacy System	11
3.2 Requirements Gathering	11
3.3 Use Cases and Stakeholders	11
4 Solution Proposal	13
4.1 Proposed System Architecture	13
4.2 Process Modelling (BPMN)	13
4.3 Data Model (ERD)	13
4.4 API and Component Design	13

4.5	Work Plan	13
5	A Short L^AT_EX Tutorial with Examples	15
5.1	Document Structure	15
5.2	Dealing with Bibliography	15
5.3	Inserting Tables	15
5.4	Importing Images	15
5.5	Floats, Figures and Captions	15
5.6	Text Formatting	17
5.7	Generating PDFs from L ^A T _E X	17
5.7.1	Generating PDFs with pdf _l atex	17
5.7.2	Dealing with Images	18
5.7.3	Dealing with Citations	18
5.7.4	Footnotes	18
5.7.5	Tables	18
5.7.6	Figures	19
5.8	Equations	19
5.9	Test for algorithms	22
	Bibliography	23

LIST OF FIGURES

5.1	A figure with two sub-figures!	16
5.2	Bitmap image (JPG/PNG)	20
5.3	Vectorial image (PDF)	21
5.4	Exemplo de utilização de <i>subbottom</i>	22

LIST OF TABLES

5.1	Test results summary.	18
-----	-------------------------------	----

ACRONYMS

ACID	Atomicity, Consistency, Isolation, Durability (<i>p. 7</i>)
AI	Artificial Intelligence (<i>pp. 4, 5</i>)
API	application programming interface (<i>pp. 5–7</i>)
BaaS	Backend-as-a-Service (<i>pp. v, 4–6</i>)
BASE	Basically Available, Soft State, Eventually Consistent (<i>p. 7</i>)
BPMN	Business Process Model and Notation (<i>p. 3</i>)
CRUD	Create, Read, Update, Delete (<i>p. 6</i>)
DI	Department of Computer Science (<i>pp. 1, 3, 5</i>)
EJS	Embedded JavaScript (<i>p. 7</i>)
ERD	Entitiy-Relationships Diagrams (<i>p. 3</i>)
FCT	NOVA School of Science and Technology (<i>pp. 1, 3, 5</i>)
HTML	HyperText Markup Language (<i>p. 7</i>)
I/O	Input/Output (<i>p. 7</i>)
JPA	Java Persitence API (<i>p. 6</i>)
LLM	large language models (<i>p. 5</i>)
NoSQL	Not only SQL (<i>pp. v, 4, 7, 8</i>)
NOVA	NOVA University Lisbon (<i>pp. 1, 3, 5</i>)
novathesis	NOVAthesis L ^A T _E X (<i>pp. i, iii</i>)

RLS	Row-Level-Security (<i>p. 6</i>)
SQL	Structured Query Language (<i>pp. v, 4, 6, 7</i>)
UX	User Experience (<i>pp. 2, 4</i>)

INTRODUCTION

1.1 Context and Background

The transition of students from academic life to the professional market or advanced research is a centerpiece of the university experience, essentially enabled through three different distinct paths at the Department of Computer Science (DI) of NOVA School of Science and Technology (FCT):

- Undergraduate Internships: Primarily offered by external companies to provide students with their first professional experience.
- Master's Dissertations: Scientific thesis typically conducted within the university, focusing on academic research under the supervision of professors.
- Engineering Projects: Practical projects proposed by external companies or the Department of Informatics itself, allowing Master's students to apply advanced engineering principles to real-world projects.

Managing those processes involves a complex coordination of multiple stakeholders: students seeking opportunities aligned with their skills and interests, professors overseeing academic quality, and external companies providing practical challenges. Within this ecosystem, a reliable digital platform is essential to act as a central hub for communication, proposal submission, and student placement.

1.2 Problem Statement

At DI@NOVA FCT, the management of internships, dissertations and engineering projects involves a multi-stage lifecycle that includes proposal submission, clarification and approval processes, making proposals available to students, report/dissertation submission, and providing documentation to juries. Currently the department relies on a legacy platform to support some of these activities, however this system presents several critical issues that hinder the efficiency of the department.

The most significant limitation is that the current platform is not being utilized for the Master's degree processes - including both scientific dissertations and engineering projects. At the moment it is exclusively used to manage undergraduate internships, meaning that the more complex workflows associated with Master's students are handled through manual, or non-standardized methods. This creates a lack of centralized data and increases the administrative burden on coordinators, professors and the secretariat.

Furthermore, even within its current scope, the legacy system suffers from several functional and technological gaps:

- **Communication Gaps and Reliability:** The current method for "Call for Proposals" is manual, requiring coordinators to send individual emails to companies, advertising the opportunity to send proposals, for each edition. There are no safeguards to ensure that these emails follow specific deliverability rules, often resulting in messages being flagged as spam. Additionally, the system lacks the flexibility to create and edit email templates, preventing coordinators from customizing communication for different editions or context.
- **Inefficient Feedback Loops:** Although companies can submit proposals, there is no integrated mechanism for coordinators to provide direct feedback on those proposals for clarification. This forces corrections to be handled outside the platform, whereas a centralized system would allow companies to correct and resubmit proposals based on coordinator comments.
- **Stale Company Data:** The existing repository contains a large amount of outdated information, including companies that no longer exist or have changed their contacts or other information. There is a lack of a "validation" status to ensure only active and verified companies can send proposals.
- **Manual Protocol Management:** For new companies that have never participated in previous editions, the process of signing the required protocols is still entirely manual. The platform should automate delivery and tracking of these legal documents as soon as a new company registers.
- **Outdated User Experience (UX):** The interface is not intuitive for external stakeholders, making it difficult for companies to register, manage their employees' accounts, or monitor the status of their proposals in real-time.

In conclusion, the necessity for a new platform arises from the need to unify all academic degrees under a single digital infrastructure, replacing manual tracking with a secure, automated system that ensures data integrity and seamless experience for all stakeholders.

1.3 Objectives

The primary objective of this dissertation is to design and develop a modernized web-based platform that unifies and automates the management and entire lifecycle of undergraduate internships, Master's dissertations, and engineering projects for DI@NOVA FCT. The projects aims to transition from a fragmented, manual system to a centralized digital ecosystem.

To achieve this goal, the following specific objectives have been defined for the initial version:

- **Process Unification:** Integrate the lifecycles of undergraduate internships and Master's degree pathways (both scientific and engineering-focused) into a single platform, replacing the current manual methods.
- **Workflow Automation:** Implement automated procedures for the "Call for Proposals student seriation (ranking), and the validation of proposals.
- **Enhanced Communication and Feedback:** Establish direct feedback loops within the platform, allowing coordinators to request clarifications on proposals and ensuring companies can resubmit corrected versions.
- **Document and Protocol Management:** Automate the delivery, tracking, and storage of legal protocols, student CVs, and recommendation letters.
- **Administrative Oversight:** Develop advanced administrative tools, including "superuser" access for student support, detailed system logs for auditing, and validation workflow for new company registrations.

1.4 Methodology

This project will follow an Agile/Scrum methodology, characterized by iterative development, frequent feedback loops [14] to ensure the final platform aligns with the university's complex requirements. The work plan is structured into several phases, prioritizing rapid prototyping followed by a transition to a high-performance custom architecture.

To achieve this goal, the following specific objectives have been defined for the initial version:

1. **Requirement Gathering and Analysis:** A through analysis of the legacy system and current manual processes was conducted to identify functional and non-functional requirements. This phase includes the definition of user roles (Admin, Coordinator, Student, Professor, Company, Secretariat) and their respective use cases.
2. **System Modeling and Design:** Utilizing Business Process Model and Notation (BPMN) to map complex workflows [12] and Entity-Relationships Diagrams (ERD)

to design the database schema [3]. This phase also involves planning the system architecture.

3. **AI-Assisted Prototyping (Vibe Coding):** The initial implementation phase will leverage Vibe Coding tools, such as Lovable, Figma Make or Google AI Studio, to rapidly generate the frontend and core functional interfaces [4]. These tools excel at creating React-based user interfaces through iterative natural language prompting. During this stage, Supabase will be utilized as Backend-as-a-Service (BaaS) to provide an immediate infrastructure for authentication, database management, and initial data structures via SQL[20]. This approach allows for the rapid fulfillment of most functional requirements while maintaining a live, testable version of the platform.
4. **Architectural Transition and Manual Development:** Once the prototype reaches a stable state and meets the primary functional requirements, the project will transition from AI-generated code to manual development. This move is essential to refine the User Experience, optimize performance, and implement custom features that go beyond the limitations of AI-driven boilerplate.
5. **Custom Backend Implementation:** To support the department's more complex business logic - such as the specialized student seriation (ranking) algorithms and robust administrative auditing - the system will transition to a dedicated custom backend, likely using Spring Boot [15]. While Supabase is effective for rapid prototyping, a Spring Boot architecture offers superior domain modeling, type safety and maintainability through its layered structure (Controller, Service, Repository). This transition ensures that the platform is not only fast to develop initially but also scalable and reliable for long-term university use.
6. **Verification and Validation:** The platform will be tested against the initial requirements, ensuring that automated emails follow specific deliverability rules avoiding being flagged as spam. Additionally, the seriation logic will be rigorously validated to ensure it correctly handles student placements and matching according to pre-defined criteria.
7. **Documentation and Evaluation:** There will be a continuous documentation of the architecture decisions, providing detailed justifications for technical choices such as the preference for SQL over NoSQL for structured academic data, and other technical choices.

STATE OF THE ART AND TECHNOLOGIES

This chapter examines the current state of software engineering practices and technologies relevant to building a modern platform for the DI at NOVA FCT. The goal is to overcome limitations of the legacy system - such as fragmented data, manual workflows, and inefficient communication - by systematically evaluating contemporary approaches to backend architectures, data persistence, frontend frameworks, and AI-assisted development workflows. Particular attention is given to the emerging paradigm of AI-assisted software development, often described as “Vibe Coding” [7] which supports rapid prototyping by allowing developers to steer large language models (LLM) through natural-language interactions instead of traditional line-by-line coding.

Modern web application stacks are characterized by a wide range of choices at each layer, from infrastructure and backend services to frontend frameworks and development workflows. In the backend, developers must choose between fully managed BaaS platforms [8] and custom backends that offer fine-grained control over domain logic, performance, and governance. On the frontend, single-page applications and component-based frameworks (such as React, Angular, or Vue) are commonly combined with RESTful or GraphQL APIs, enabling decoupled client-server architectures and iterative delivery of new features.

Beyond classical architectures, AI-assisted development tools and practices are reshaping how software is designed and implemented. Under the “Vibe Coding” paradigm, developers describe desired functionality to an LLM, which generates and refines code iteratively, prioritizing rapid experimentation and high-level intent over manual code authoring. This approach aligns with agile or small teams to assemble complex prototypes quickly, though it raises questions about maintainability, security, and auditability in production systems. [13]

2.1 Backend Architectures and BaaS

The choice of backend architecture directly influences development speed, scalability, operational complexity, and the ability to encode complex business rules such as student

seriation and multi-role access control. For this project, two main paradigms are considered: Backend-as-a-Service solutions, which offer ready-made infrastructure and managed services, and custom backends built with mature frameworks such as Spring Boot and Express.js, which provide full control over domain modeling, integration patterns, and deployment environments. [2]

BaaS platforms typically optimize for rapid time-to-market by abstracting away server provisioning, database management, and authentication, making them well-suited for early-stage prototyping and small teams [1]. In contrast, custom backends allow tailoring of data models, ownerships of the entire codebase, and fine-grained observability, which becomes critical for institutional applications that must satisfy compliance, long-term maintainability, and integration with existing academic processes.

2.1.1 Supabase (Backend-as-a-Service)

Supabase is an open-source BaaS stack that builds on PostgreSQL and exposes automatically generated REST APIs through PostgREST [18], significantly reducing boilerplate CRUD implementation effort. By providing an integrated suite of tools - including database, authentication, storage, and real-time capabilities - Supabase enables rapid development cycles while leveraging a relational data model that is compatible with established SQL tooling and practices. [20]

A central feature of Supabase is its built-in authentication system, which supports user registration, password-based login, magic links, and role-based access control[16], simplifying the enforcement of different permission profiles for students, companies, professors and administrators. Row-Level-Security (RLS) policies can be defined directly at the database level, ensuring that authorization rules are enforced close to the data and cannot be bypassed by misconfigured API clients.[19] For logic that goes beyond standard CRUD operations - such as orchestrating notification workflows or managing student seriation - Supabase offers serverless “Edge Functions”, typically written in TypeScript or JavaScript, which run in a managed environment near end users to minimize latency. [17]

2.1.2 Custom Backend: Spring Boot

Spring Boot is a widely adopted framework for building production-grade Java or Kotlin backends and is especially suitable for complex, domain-driven systems that require strict typing, layered architectures, and extensive integration capabilities. Its opinionated configuration model and extensive ecosystem (Spring Data, Spring Security, Spring Cloud) allow developers to structure applications into controllers, services, and repositories, facilitating clear separation of concerns and maintainable domain models. [15]

For academic workflows that must support student ranking algorithms, complex eligibility rules, and auditable decision-making processes, the type safety and testability provided by Spring Boot are particularly advantageous. The framework integrates with JPA and tools like Hibernate Envers to support automatic auditing of entity changes [15,

10], which helps track user actions, state transitions, and login-related events - capabilities that are often required for administrative oversight and internal compliance in university environments.

2.1.3 Custom Backend: Express.js (Node.js)

Express.js is a minimalist web framework for Node.js that offers a lightweight and flexible foundation for building HTTP APIs using JavaScript or TypeScript [6]. Its event-driven, non-blocking I/O model allows a single process to handle many concurrent connections efficiently, making it well-suited for notification-heavy workloads [11] such as broadcasting “Call for Proposals” or status updates to large groups of students.

The Node.js ecosystem provides a rich set of libraries for templated email generation and delivery, including tools such as Handlebars or EJS to render dynamic HTML email bodies [9, 5] for coordinators and administrators. Combined with tasks schedulers or message queues, Express-based backends can orchestrate asynchronous communications and background jobs [11], helping address a key functional gap of legacy systems that rely on manual email workflows and ad hoc templates.

2.2 Data Persistence: SQL vs. NoSQL

The selection of a data persistence model is a fundamental architectural decision that dictates how the system ensures data integrity, scalability, and retrieval efficiency. Given the complex ecosystem of the thesis and internships management, which involves multiple stakeholders (Students, Professors, Companies, Secretariat, etc..) and multi-stage lifecycle workflows for academic projects, this section evaluates the trade-offs between Relational (SQL) and Non-Relational (NoSQL) databases.

The choice between SQL and NoSQL has complex implications for institutional systems. Relational databases prioritize data integrity and consistency through Atomicity, Consistency, Isolation, Durability (ACID) compliance, while Non-Relational systems typically follow the Basically Available, Soft State, Eventually Consistent (BASE) model, trading immediate consistency for horizontal scalability and schema flexibility. For academic management platforms where data accuracy are indispensable, this distinction is critical.

2.2.1 The Relational Model (SQL)

Relational databases structure data into tables with predefined schemas, utilizing Foreign Keys to establish strict relationships between entities. This approach provides fundamental guarantees about data quality and consistency that are essential for institutional systems.

- **Structured Data and Integrity:** The academic context requires rigorous data consistency. For instance, a Candidacy must strictly link to an existing Student and a valid Proposal. The relational model enforces referential integrity through primary

and foreign key constraints, preventing "orphaned" records - a situation where a child record references a non-existent parent record. This is particularly critical in the DI management platform, where deleted Companies should not retain active Proposals, and Students cannot be assigned to non-existent projects.

- Referential integrity mechanism ensure that every foreign key in a child table corresponds to a valid primary key in the parent table, maintaining logical consistency across the entire database. Beyond preventing data anomalies, referential integrity supports cascading updates and deletions, ensuring that changes propagate correctly across related entities.
- Complex Querying: The platform requires complex join operations to generate views - for example, listing all proposals for a specific edition filtered by a company's validation status, or retrieving all students that are candidates to a particular internship. SQL excels at these complex relationships through its mature and optimized query language, which can efficiently traverse multiple table relationships in a single query.
- PostgreSQL: This project will initially utilize PostgreSQL (via Supabase for rapid prototyping). Beyond standard SQL features, PostgreSQL offers support for complex data types (JSON/JSONB, arrays, enums) and Stored Procedures. This allows the system to handle semi-structured data - such as like flexible feedback forms or email template configurations - while maintaining the benefits of a relational schema for academic entities.

2.2.2 The Non-Relational Model (NoSQL)

NoSQL databases (e.g., document stores like MongoDB or key-value systems like Redis) offer schema flexibility and horizontal scalability, often at the cost of immediate consistency (ACID properties)

- Flexibility vs Structure: While NoSQL allows for rapid iteration of data models without schema migrations, the academic domain is highly structured. Entities such as Users, Editions, Proposals, Companies and Students have well-defined attributes that rarely change dynamically or unexpectedly. The governance requirements of an academic institution demand clear, predictable data structures rather than evolving, loosely-typed records.
- Consistency Challenges: In workflows like Student Seriation, data consistency is essential. A NoSQL approach, which often relies on eventual consistency, could introduce race conditions where a student is assigned to a project that is no longer available or where multiple simultaneous requests lead to conflicting state changes. The BASE model prioritises availability and partition tolerance over strict consistency,

which is acceptable for read-heavy analytics systems but unsuitable for transactional workloads that require immediate correctness.

2.2.3 Justification for the Chosen Approach (PostgreSQL)

2.3 Frontend Frameworks and User Experience

2.4 AI-Assisted Development (Vibe Coding)

REQUIREMENTS AND CURRENT SYSTEM ANALYSIS

- 3.1 Analysis of the Legacy System
- 3.2 Requirements Gathering
- 3.3 Use Cases and Stakeholders

SOLUTION PROPOSAL

- 4.1 Proposed System Architecture**
- 4.2 Process Modelling (BPMN)**
- 4.3 Data Model (ERD)**
- 4.4 API and Component Design**
- 4.5 Work Plan**

A SHORT L^AT_EX TUTORIAL WITH EXAMPLES

This Chapter aims at exemplifying how to do common stuff with L^AT_EX. We also show some stuff which is not that common! ;)

Please, use these examples as a starting point, but you should always consider using the *Big Oracle* (aka, Google, your best friend) to search for additional information or alternative ways for achieving similar results.

5.1 Document Structure

5.2 Dealing with Bibliography

Citing something online [`wiki:shuntingyard`, `flex`, `bison`].

5.3 Inserting Tables

5.4 Importing Images

5.5 Floats, Figures and Captions

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

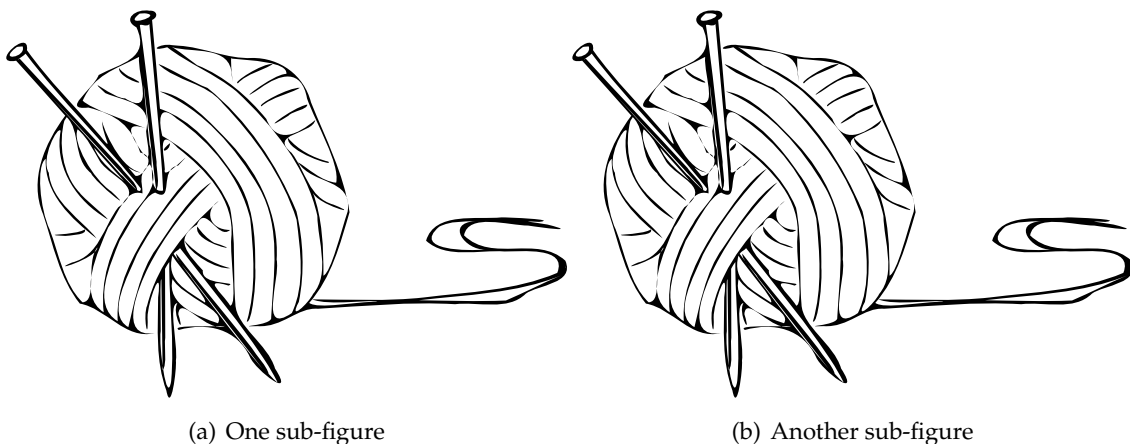


Figure 5.1: A figure with two sub-figures!

And this is a small text that references the Figure 5.1 and its Subfigures 5.1(a) and 5.1(b).

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan

eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

5.6 Text Formatting

5.7 Generating PDFs from L^AT_EX

5.7.1 Generating PDFs with pdf_latex

You may create PDF files either by using `latex` to generate a DVI file, and then use one of the many DVI-2-PDF converters, such as `dvipdfm`.

Alternatively, you may use `pdflatex`, which will immediately generate a PDF with no intermediate DVI or PS files. In some systems, such as Apple, PDF is already the default format for L^AT_EX. I strongly recommend you to use this approach, unless you have a very good argument to go for `latex + dvipdfm`.

A typical pass for a document with figures, cross-references and a bibliography would be:

```
$ pdflatex template
$ bibltex template
$ pdflatex template
$ pdflatex template
```

You will notice that there is a new PDF file in the working directory called `template.pdf`. Simple :)

Please note that, to be sure all table of contents, cross-references and bibliographic citations are up-to-date, you must run `latex` once, then `bibltex`, and then `latex` twice.

5.7.2 Dealing with Images

You may process the same source files with both `latex` or `pdflatex`. But, if your text include images, you must be careful. `latex` and `pdflatex` accept images in different (exclusive) formats. For `latex` you may use EPS ou PS figures. For `pdflatex` you may use JPG, PNG or PDF figures. I strongly recommend you to use PDF figures in vectorial format (do not use bitmap images unless you have no other choice).

5.7.3 Dealing with Citations

Para fazer citações, deverá usar-se a chave da referência no ficheiro BibTeX. Se for uma única referência [Artho04], usar um “~” para ligar o `\cite{...}` à palavra que o precede (...referência~\cite{Artho04}). Caso queira fazer múltiplas citações [Shavit95, Silberschatz06, Moss85], deverá agrupá-las dentro de um único `\cite{...}`.

Note que o ficheiro de bibliografia pode ter tantas entradas quantas quiser. Apenas aquelas cuja chave seja referenciada no texto é que serão incluídas na listagem de bibliografia.

5.7.4 Footnotes

Footnotes¹ will be numbered and shown in the bottom of the page.

5.7.5 Tables

The Table 5.1 illustrates some important concepts associated with table construction:

- i) Do not use vertical lines;
- ii) The caption should be above the table;
- iii) Use the macros `\toprule`, `\midrule` and `\bottomrule` to make the top, inner and bottom horizontal lines, respectively.

Table 5.1: Test results summary.

Test	Anomalies	Warnings	Correct	Categories	Missed
Connection [Beckman08]	2	2	1	C	1
Coordinates’03 [Artho03]	1	4	1	2B, 1C	0
Local Vari- able [Artho03]	1	2	1	A	0
NASA [Artho03]	1	1	1	—	0
Coordinates’04 [Artho04]	1	4	1	3C	0
Buffer [Artho04]	0	7	0	2A, 1B, 2C, 2D	0
Double- Check [Artho04]	0	2	0	1A, 1B	0

¹This is a simple footnote.

StringBuffer [Flanagan0	1	0	0	—	1
Account [Praun03]	1	1	1	—	0
Jigsaw [Praun03]	1	2	1	C	0
Over-reporting [Praun03]	0	2	0	1A, 1C	0
Under-reporting [Praun03]	1	1	1	—	0
Allocate Vec-tor [IBM-Rep]	1	2	1	C	0
Knight Moves [Beckman08]	1	3	1	2B	0
Total	12	33	10	5A, 6B, 10C, 2D	2

5.7.6 Figures

The images inserted in the document must be of good quality, preferably in vector format (vector PDF) and not in *bitmap* (PNG, JPG, etc.). *bitmap* images (Figure 5.2) do not scale well and have negative effects on the quality of your document. On the other hand, *vector* images Figure 5.3 scale as much as necessary without degrading the quality of the image.

You should only use *screenshots* for your plots, charts, etc, if you absolutely have no other alternative. Instead of generating a *screenshot*, try using a virtual PDF printer and printing to a PDF file. As a general rule, you will get a vector PDF. Even if your PDF contains images, they will always be of higher or equal quality than what you would get with a *screenshot*.

To combine several figures into a single one... You can then reference the set as Figure 5.4 or the sub-figures separately as 5.4(a) and 5.4(b).

5.8 Equations

LaTeX is a powerful tool for writing in a mathematical style. It allows you to insert formulas into the text, such as this: $ax^2 + bx + c = 0$. It also allows formulas to be highlighted on a separate line and centered on the page.

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

or numbered

$$e = mc^2 \tag{5.1}$$

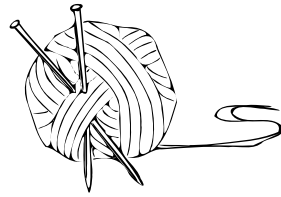
which can latter be referenced as equation 5.1



Figure 5.2: Bitmap image (JPG/PNG)



Figure 5.3: Vectorial image (PDF)



(a) Novelo de lã



(b) Tempestade com neve

Figure 5.4: Exemplo de utilização de *subbottom*

5.9 Test for algorithms

Uncomment the algorithms source below and add the following to file “5_packages.tex”

```
\usepackage{algorithm2e}
\RestyleAlgo{ruled}
```

and uncomment

```
\ntaddlistof{listofalgorithms}
```

in file “8_list_og.tex”.

BIBLIOGRAPHY

- [1] AlSalah, Point of SaaS. *Benefits and Drawbacks of Using Backend as a Service*. 2025. URL: <https://pointofsaas.com/benefits-and-drawbacks-of-using-backend-as-a-service-3/> (cit. on p. 6).
- [2] B. Biskupski and A. Pytko-Włodarczyk. *Backend as a Service (BaaS) vs. custom backend: Which one to choose and why?* Accessed: 2026-01-17. 2019. URL: <https://www.merixstudio.com/blog/backend-service-baas-vs-custom-backend> (cit. on p. 6).
- [3] P. P. Chen. “The Entity-Relationship Model: Toward a Unified View of Data”. In: *ACM Transactions on Database Systems* 1.1 (1976), pp. 9–36 (cit. on p. 4).
- [4] W. contributors. *Vibe coding - chat based AI-assisted software development definition*. Accessed: 2026-01-17. 2026. URL: https://en.wikipedia.org/wiki/Vibe_coding (cit. on p. 4).
- [5] *Embedded JavaScript templating Documentation*. EJS. 2026. URL: <https://ejs.co/#docs> (cit. on p. 7).
- [6] *Express.js Documentation*. Express.js Foundation. 2026. URL: <https://expressjs.com/> (cit. on p. 7).
- [7] Figma. *What is Vibe Coding? Everything You Need to Know*. 2025. URL: <https://www.figma.com/resource-library/what-is-vibe-coding/> (cit. on p. 5).
- [8] O. Glib. *Backend as a Service use Cases: How to apply*. Accessed: 2026-01-17. 2024. URL: <https://acropolium.com/blog/baas-use-cases/> (cit. on p. 5).
- [9] *Handlebars.js Documentation*. Handlebars.js. 2026. URL: <https://handlebarsjs.com/> (cit. on p. 7).
- [10] Hibernate Community. *Chapter 15. Envers*. 2013. URL: <https://docs.hibernate.org/orm/4.3/devguide/en-US/html/ch15.html> (cit. on p. 7).
- [11] *Node.js Documentation*. Node.js Foundation. 2026. URL: <https://nodejs.org/en/docs/> (cit. on p. 7).

- [12] Object Management Group. *Business Process Model And Notation (BPMN) Version 2.0*. OMG Document Number: formal/2011-01-03. Version 2.0. Object Management Group, 2011 (cit. on p. 3).
- [13] J. Schibelli. *Vibe Coding: How AI is Transforming Software Development*. Accessed: 2026-01-17. 2025. URL: <https://dev.to/johnschibelli/the-emergence-of-vibe-coding-how-ai-is-revolutionizing-software-development-4275> (cit. on p. 5).
- [14] K. Schwaber and J. Sutherland. *The 2020 Scrum Guide*. Accessed: 2026-01-17. 2020. URL: <https://scrumguides.org/> (cit. on p. 3).
- [15] Spring Team. *Spring Boot Reference Documentation*. 3.x. Version 3.x. VMware, Inc. 2026. URL: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/> (cit. on pp. 4, 6).
- [16] Supabase. *Auth Overview*. 2026. URL: <https://supabase.com/docs/guides/auth> (cit. on p. 6).
- [17] Supabase. *Edge Functions*. 2026. URL: <https://supabase.com/docs/guides/functions> (cit. on p. 6).
- [18] Supabase. *REST API Overview*. 2026. URL: <https://supabase.com/docs/guides/api#rest-api-overview> (cit. on p. 6).
- [19] Supabase. *Row Level Security*. 2026. URL: <https://supabase.com/docs/guides/database/postgres/row-level-security> (cit. on p. 6).
- [20] Supabase. *Supabase: The Open Source Firebase Alternative*. <https://github.com/supabase/supabase>. Project README. 2019 (cit. on pp. 4, 6).



